

1. Inicialización del Mando

```
pros::Controller master(pros::E_CONTROLLER_MASTER);
```

- **pros::Controller:** Esta clase representa el mando.
- **master:** Es el nombre por defecto que se le da a la variable de tipo Controller.
- **pros::E_CONTROLLER_MASTER:** Esto indica que se va a usar el Joystick principal del mando para los movimientos.
 - **pros::E_CONTROLLER_PARTNER** es para implementar dos joysticks.

2. Inicialización de un Motor

```
pros::Motor leftMotor1(2, pros::v5::MotorGears::green, pros::v5::MotorUnits::degrees);
```

- Argumentos (Puerto en el que esta conectado al Cerebro, Tipo de engranajes, Unidades)

Parámetro	Descripción	Ejemplo usado
2	Puerto donde está conectado al Cerebro.	Puerto 2
pros::v5::MotorGears::green	El tipo de engranajes del motor: - green: engranaje estándar 100 RPM - red: engranaje de torque (36 RPM) - blue: engranaje velocidad (200 RPM)	Engranajes de 100 RPM
pros::v5::MotorUnits::degrees	Unidades para medir la posición/velocidad del motor	Grados (rotación del eje)

```
move(int voltage):
```

- Envía potencia al motor (rango: -127 a 127).

```
tare_position():
```

- Reinicia la posición del encoder a cero.

```
get_position():
```

- Devuelve la posición del motor en grados desde el último reinicio.

3. Comandos para usar la Consola en Visual

Pros clear

- Sirve para reiniciar el proyecto y eliminar cada uno de los elementos o archivos que se habian creado en el proyecto

Pros make

- Compila el proyecto para antes de subirlo al Cerebro

Pros upload

- Sube el codigo al Cerebro dependiendo el slot en donde este.

pros upload --slot=N

- Sube el codigo al Cerebro en el Slot N que se indique.

pros make && pros upload --slot=0 --disable-watchdog

- Comando para pasar los analisis y subir rapido el codigo

pros c clean

- Limpia archivos compilados.

pros c build

- Reconstruye el proyecto desde cero.

Cada uno de los comandos es fundamental dado que para subir los proyectos al cerebro y las diferentes acciones. Toca hacerlo full por consola.

4. Esperas utilizando Delay

pros::delay(ms):

Detiene la ejecucion del programa por la cantidad de *ms* milisegundos que se especifiquen.

5. Depuración

```
printf("Valor: %d", variable);
```

Permite imprimir valores en la consola.

- La terminal de PROS muestra la salida en tiempo real.

5. Estructura Proyecto

- **main.h**: Cabecera principal del proyecto. Incluye las declaraciones necesarias generales que van a ser implementadas en el proyecto.
- **globals.hpp y globals.cpp**: Archivos donde se declaran y definen los objetos globales como motores y sensores.
- **drive.cpp**: Archivo donde se implementan cada una de las funciones de movimiento (driveForward(), turnRight(), turnLeft())

SENSOR INERCIA

1. Funcionamiento

El sensor esta enfocado en medir:

- **Giros**: Cuánto ha girado el robot, dependiendo de una posicion inicial.
- **Dirección**: Indica hacia dónde está mirando, según la inicialización.
- **Inclinaciones**: Inclinación que tiene el robot hacia adelante, atrás o a los lados.
- **Aceleración**: Velocidad del robot en un momento determinado.

2. Inicializar Sensor

```
#define IMU_PORT 1
```

```
pros::Imu imu(IMU_PORT);
```

- El IMU_PORT es el número de puerto al cual está conectado el sensor al cerebro.

3. Calibración

```
imu.reset();
```

```
while (imu.is_calibrating()) {
```

```
    pros::delay(10);
```

```
}
```

- `imu.reset()` empieza a calibrar el sensor (tarda aprox 2 seg) y retorna inmediatamente.
- `imu.reset(true)` bloquea el hilo (tarea) hasta que el sensor esté listo.
- `imu.reset(false)` calibra el sensor en segundo plano.

4. Funciones para Obtener Información

1. get_rotation

- **¿Qué hace?:** Da la rotación total en grados que ha girado el robot, alrededor del eje Z (eje Vertical). Dado que es total el valor que retorna esta función puede ser mayor a 360° si el robot da más de una vuelta.
- **¿Cuándo usarlo?:** Útil para saber cuántas vueltas ha dado el robot o para medir giros grandes.
- **Implementación:**

```
double rotacion = miSensor.get_rotation();
```

2. get_heading

- **¿Qué hace?:** Incide hacia dónde está mirando el robot, respecto al plano horizontal xy. Funciona como una brújula (de 0° a 360°), siendo 0° la dirección inicial, y este gira en sentido horario.
- **¿Cuándo usarlo?:** Permite una ubicación en la cancha para poder apuntar a un objetivo específico o conocer cuanto debe realizar de giro.
- **Implementación:**

```
double direccion = miSensor.get_heading();
```

3. get_pitch

- **¿Qué hace?:** Retorna la inclinación del robot hacia adelante o atrás, permitiendo realizar corrección si el robot se está levantando del suelo.
- **¿Cuándo usarlo?:** Para evitar que se incline al acelerar o frenar de una manera fuerte.
- **Implementación:**

```
double inclinacion = miSensor.get_pitch();
```

4. get_roll

- **¿Qué hace?:** Mide la inclinación del robot hacia los lados (izquierda o derecha), permitiendo realizar corrección si el robot se está levantando del suelo al girar.
- **¿Cuándo usarlo?:** Para que el robot siempre este en contacto con el suelo independientemente de la velocidad con la cual gire o choque.
- **Implementación:**

```
double balanceo = miSensor.get_roll();
```

5. get_yaw (Demasiado Difícil de Programar)

- **¿Qué hace?:** Implementa ángulos de Euler para trabajar al tiempo con los 3 ejes (x y z).
- **Implementación:**

```
double yaw = miSensor.get_yaw();
```

6. get_accel

- **¿Qué hace?:** Mide la aceleración del robot en los ejes X, Y y Z. La aceleración se mide en unidades de "g" (gravedad).
- **¿Cuándo usarlo?:** Para detectar movimientos bruscos, al acelerar o frenada rápidamente.
- **Implementación:**

```
pros::c::imu_accel_s_t accel = miSensor.get_accel();
```

7. get_gyro_rate

- **¿Qué hace?:** Mide la velocidad a la que el robot está girando en los ejes X, Y y Z.
- **¿Cuándo usarlo?:** Detecta un giro rápido y permite estabilizar el robot.
- **Implementación:**

```
pros::c::imu_gyro_s_t gyro = miSensor.get_gyro_rate();
```

5. Funciones para Ajustar Referencias: "Tare"

Cada una de las funciones Tare permite resetear las lecturas anteriores que haya hecho el sensor y tome la posición actual como el nuevo punto de partida (0°).

1. tare_rotation

- **¿Qué hace?:** Devuelve la rotación acumulada a 0° y vuelve a contar todo desde este punto.
- **¿Cuándo usarlo?:** Antes de realizar un giro, para medir cuántos grados gira el robot desde ese momento.
- **Implementación:**

```
miSensor.tare_rotation();
```

2. tare_heading

- **¿Qué hace?:** Establece la dirección actual como 0°.
- **¿Cuándo usarlo?:** Cuando se quiera que la dirección actual sea el nuevo "frente" (giros).
- **Implementación:**

```
miSensor.tare_heading();
```

3. tare

- **¿Qué hace?:** Reinicia todas las orientaciones (rotación, heading, pitch, roll, yaw) a 0°.
- **¿Cuándo usarlo?:** Al iniciar una nueva tarea o después de recalibrar.
- **Implementación:**

```
miSensor.tare();
```

4. Otras funciones Tare:

Reinician cada uno de los ángulos de manera individual:

- `tare_pitch()` // Reinicia el angulo en el eje Y
- `tare_roll()` // Reinicia el angulo en el eje X
- `tare_yaw()` // Reinicia el angulo en el eje Z

6. Funciones para Establecer Valores: "Set"

Si sabes exactamente en qué posición está el robot, puedes decirle al sensor cuál es esa posición usando las funciones "set".

1. set_heading

- **¿Qué hace?:** Establece la dirección actual a un valor específico que se envía como parametro de la función.
- **¿Cuándo usarlo?:** Cuando se tiene certeza del angulo en el cual se encuentra el robot y se cree que esta descalibrado.
- **Implementación:**

```
miSensor.set_heading(90.0);
```

2. set_rotation

- **¿Qué hace?:** Establece la rotación acumulada a un valor específico.
- **¿Cuándo usarlo?:** Si se quiere continuar una medición desde un punto conocido.
- **Implementación:**

```
miSensor.set_rotation(180.0);
```

Otras funciones de set:

set_yaw(), set_pitch(), set_roll(): Ajustan cada ángulo individualmente.

set_euler(): Ajusta pitch, roll y yaw al mismo tiempo.